

LED 补光网关 V1.5 HTTP 接口指令手册

适用版本： LED 网关 V1.5（8 台双通道驱动器，16 路）

工控机地址： 192.168.123.103（当前局域网地址，变化后需替换）

HTTP 端口： 8090

接口前缀： /api/v1/led

编写依据： V1.5 工程 server.py、light_modbus.py、web/app.js。

1. 系统和设备

本机或局域网客户端 → HTTP 网关 → Linux 工控机 /dev/ttyUSB0 → USB-RS485 → LED 驱动器。底层为 Modbus RTU，9600 baud、8N1。

光谱组	地址（各自独立控制）	CH1	CH2
1	0x96、0x9A	660 nm	395 nm
2	0x97、0x9B	450 nm	530 nm
3	0x98、0x9C	630 nm	430 nm
4	0x99、0x9D	730 nm	全光谱

注意： 同组光谱相同，但两台驱动器不会自动联动。每个地址包含 CH1、CH2 两路。光强参数为 **0~255 的整数**；0 为关闭，255 为协议最大设定值，不代表实测辐照度。

2. 服务启动与认证

2.1 工控机局域网启动（推荐）

在工控机终端：

```
cd ~/project/led_linux_gateway
read -rsp "请输入 LED API Key: " LED_API_KEY
echo
export LED_API_KEY
LED_SERIAL_PORT=/dev/ttyUSB0 \
LED_HTTP_HOST=0.0.0.0 \
LED_HTTP_PORT=8090 \
python server.py
```

服务端对外监听（0.0.0.0）时，必须配置有效 **LED_API_KEY**；未配置会拒绝启动。**不要将真实密钥写入文档、代码库或截图。**

本机仅供调试时可使用 **LED_HTTP_HOST=127.0.0.1**；如没有设置密钥，本地接口可无密钥访问。

2.2 请求认证

启用密钥后，所有 /api/v1/led/* 请求必须携带：

X-API-Key: <你的真实密钥>

工控机新开终端，可安全输入密钥：

```
read -rsp "请输入 LED API Key: " LED_KEY
echo
```

后文示例使用 `$LED_KEY`。浏览器网页在 <http://192.168.123.103:8090/> 中填写相同密钥。

2.3 地址与网页

- 工控机本地：<http://127.0.0.1:8090/>
- 同 WiFi 的电脑：<http://192.168.123.103:8090/>
- Swagger 接口文档：<http://192.168.123.103:8090/docs>
- 网页广播 V1.5：发送一次 → 等待约 3 秒 → 回读全部 → 对异常设备最多额外重试两轮；**此等待/重试属于网页逻辑，直接调用广播 HTTP 接口不会自动执行回读确认。**

3. HTTP 接口总览

方法	路径	作用
GET	/api/v1/led/health	查看 HTTP 服务及串口打开状态
GET	/api/v1/led/read?busAddress=0x96	读取指定驱动器两路光强
GET	/api/v1/led/read-all	顺序读取 8 台设备
POST	/api/v1/led/set	写入指定驱动器的 CH1、CH2
POST	/api/v1/led/broadcast/set	通过 RS485 广播设置所有设备的 CH1、CH2

POST 请求头：Content-Type: application/json。GET 无需请求体。地址仅接受 0x96 至 0x9D 两位十六进制字符串（大小写均可）；数值只接受整数 0..255。

4. 接口详细说明与命令

4.1 GET /health — 服务健康状态

```
curl -sS -H "X-API-Key: $LED_KEY" \
  "http://127.0.0.1:8090/api/v1/led/health"
```

可能返回：

```
{"ok":true,"serialConnected":true}
```

`serialConnected` 只表示网关串口当前已打开；`false` 可能是尚未触发首次串口访问，**不等于八台设备都离线**。此接口不逐台检测设备。

4.2 GET /read — 读取指定设备

```
curl -sS -H "X-API-Key: $LED_KEY" \
  "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x96"
```

示例响应：

```
{"busAddress":"0x96","ch1":30,"ch2":30}
```

`busAddress` 必填；设备不在线或响应校验失败时通常返回 HTTP 502，串口打不开返回 503。

4.3 GET /read-all — 读取八台设备

```
curl -sS -H "X-API-Key: $LED_KEY" \
  "http://127.0.0.1:8090/api/v1/led/read-all"
```

成功时 `devices` 包含 `0x96`、`0x97`、`0x98`、`0x99`、`0x9A`、`0x9B`、`0x9C`、`0x9D` 八项，每项有 `busAddress`、`ch1`、`ch2`、`ok`。

```
{
  "devices": [
    {"busAddress": "0x96", "ch1": 30, "ch2": 30, "ok": true},
    {"busAddress": "0x97", "ch1": null, "ch2": null, "ok": false, "error": "RS485 等待响应超时"}
  ]
}
```

以上仅展示两条记录来说明结构，实际响应通常包含八条。**注意：**即使某台 `ok:false`，此接口仍可能整体返回 HTTP 200，调用方必须逐条检查 `ok`。

4.4 POST /set — 设置单台两路光强

```
curl -sS -X POST \
  "http://127.0.0.1:8090/api/v1/led/set" \
  -H "X-API-Key: $LED_KEY" \
  -H "Content-Type: application/json" \
  -d '{"busAddress": "0x96", "ch1": 30, "ch2": 20}'
```

成功响应：

```
{"ok": true, "busAddress": "0x96", "ch1": 30, "ch2": 20}
```

重要：该接口一次写入 CH1、CH2 两路。如果只想改 CH1，请先用 `/read` 读取 CH2 原值，再在 `/set` 中原样传回 CH2，避免误改另一通道。成功响应表示后端写入操作完成；如需确认设备状态，请再 `/read` 回读。

示例：设置 `0x9D` 两路为 0：

```
curl -sS -X POST \
  "http://127.0.0.1:8090/api/v1/led/set" \
  -H "X-API-Key: $LED_KEY" \
  -H "Content-Type: application/json" \
  -d '{"busAddress": "0x9D", "ch1": 0, "ch2": 0}'
```

4.5 POST /broadcast/set — 广播设置全部设备

广播不需要 `busAddress`，会向 Modbus 广播地址 `0x00` 发送一条双通道写入命令；所有接受广播的设备可能同时改变亮度。

广播到 30/30（现场确认允许开灯后执行）：

```
curl -i -X POST \
  "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
  -H "X-API-Key: $LED_KEY" \
  -H "Content-Type: application/json" \
  -d '{"ch1": 30, "ch2": 30}'
```

广播全部关闭：

```
curl -sS -X POST \
  "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
  -H "X-API-Key: $LED_KEY" \
  -H "Content-Type: application/json" \
  -d '{"ch1": 0, "ch2": 0}'
```

广播协议最大值（谨慎执行）：

```
curl -sS -X POST \
  "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
  -H "X-API-Key: $LED_KEY" \
```

```
-H "Content-Type: application/json" \  
-d '{"ch1":255,"ch2":255}'
```

成功响应示例：

```
{  
  "ok":true,  
  "scope":"BROADCAST",  
  "ch1":30,  
  "ch2":30,  
  "note":"broadcast may have no per-device ack"  
}
```

HTTP 200 / `ok:true` 仅表示后端发送流程成功，并不代表 8 台设备均已确认执行。广播没有逐台确认应答。建议等待后再读取：

```
sleep 3  
curl -sS -H "X-API-Key: $LED_KEY" \  
  "http://127.0.0.1:8090/api/v1/led/read-all"
```

逐条检查 `ok:true`，且 CH1、CH2 均等于广播目标值。若部分设备超时，延迟后对异常地址用 `/read` 重试；**不要因为回读失败而自动重复广播写入。**

5. HTTP 状态码和常见错误

HTTP 状态	可能含义	建议
200	请求完成；read-all 仍需检查逐台 ok	检查 JSON 数据
401	X-API-Key 缺失或不匹配	检查客户端输入与服务端密钥
422	地址、字段、整数范围或 JSON 参数无效	检查 busAddress、ch1、ch2
502	单台 Modbus 通信/校验失败	检查地址、RS485 总线、设备供电
503	串口打开或串口收发异常	检查 /dev/ttyUSB0、权限、占用
500	未处理的服务端异常	查看工控机 server.py 终端日志

`/read-all` 会捕获逐台通信异常并在 `devices` 里返回 `ok:false`，不保证整个请求以 502 返回。

6. 在 Windows 本机通过 HTTP 控制

你的工控机 IP 是 `192.168.123.103`。Windows PowerShell 可用 `curl.exe`（避免与 PowerShell 旧版别名混淆）。示例：

```
$LED_KEY = Read-Host "请输入 LED API Key"  
curl.exe -H "X-API-Key: $LED_KEY" "http://192.168.123.103:8090/api/v1/led/read-all"  
设置单台：
```

```
curl.exe -X POST "http://192.168.123.103:8090/api/v1/led/set" \  
-H "X-API-Key: $LED_KEY" \  
-H "Content-Type: application/json" \  
-d '{"busAddress":"0x96","ch1":30,"ch2":20}'
```

PowerShell 对引号转义因版本而异；如上例出现 JSON 解析问题，建议使用 `Invoke-RestMethod`：

```
$body = @{"busAddress"="0x96"; ch1=30; ch2=20} | ConvertTo-Json -Compress  
Invoke-RestMethod -Uri "http://192.168.123.103:8090/api/v1/led/set" \  
-Method Post -Headers @{"X-API-Key"=$LED_KEY} \  
-ContentType "application/json" -Body $body
```

也可以直接使用网页 `http://192.168.123.103:8090/`，输入密钥后操作。

7. 推控制流程

1. 启动服务并确认网络可访问。
2. 调用 `/health`，再调用 `/read-all`，确认设备状态。
3. 单台调光时使用 `/set`，随后 `/read` 核验。
4. 全局调光时使用 `/broadcast/set`，等待约 3 秒，再 `/read-all` 逐台核验。
5. 设备通信失败时仅重试读取，避免误重复写入。
6. 结束后根据现场需求恢复光强或广播关闭。

8. 安全和维护

- API Key 仅用于 HTTP 鉴权，不加密 HTTP 流量。建议仅在可信局域网使用，限制防火墙访问来源，不要映射公网端口。
- 启用 `LED_HTTP_HOST=0.0.0.0` 时必须提供真实密钥。
- 不要把密钥放在公开仓库、截图或共享文档中；如泄露应轮换。
- 395 nm 光源可能带来眼部风险，避免直视灯具。
- `/docs` 为接口说明页；真实写入会改变灯光，请谨慎使用 Try it out。
- 本手册的响应示例仅用于解释字段，实际设备数值以回读结果为准。

版本说明： V1.5 的广播延时、回读和有限重试主要由网页 `web/app.js` 执行。后端五个 HTTP 接口沿用 V1.4/V1.2 的实现；直接用 curl 调用广播时，需要调用方自己完成等待和回读。